Module 20: Mobile DevOps Fundamentals

- CI/CD Pipelines, Build Automation, Secrets and Signing, Versioning, Environments, and Artifact Repositories
- Enterprise Mobile Application Development Bootcamp, AI-Integrated Edition



©Logical Imagination Group LLC

What We Will Cover

- Part 1: CI/CD Foundations. Pipelines as graphs, reproducible builds and caching, tests and quality gates, secrets, signing, and pipeline security
- Part 2: Releases, Environments, and Artifacts. Versioning and release trains, configuration and promotion, artifact repositories, delivery metrics and the release decision



©Logical Imagination Group LLC

Part 1: CI/CD Foundations: Pipelines, Builds, Tests, and Secrets

- 1 CI/CD Concepts and the Mobile Delivery Pipeline
- 2 Build Automation: Reproducible Builds and Caching
- 3 Automated Testing in the Pipeline and Quality Gates
- 4 Secrets, Signing, and Pipeline Security



©Logical Imagination Group LLC

Lesson 1: CI/CD Concepts and the Mobile Delivery Pipeline

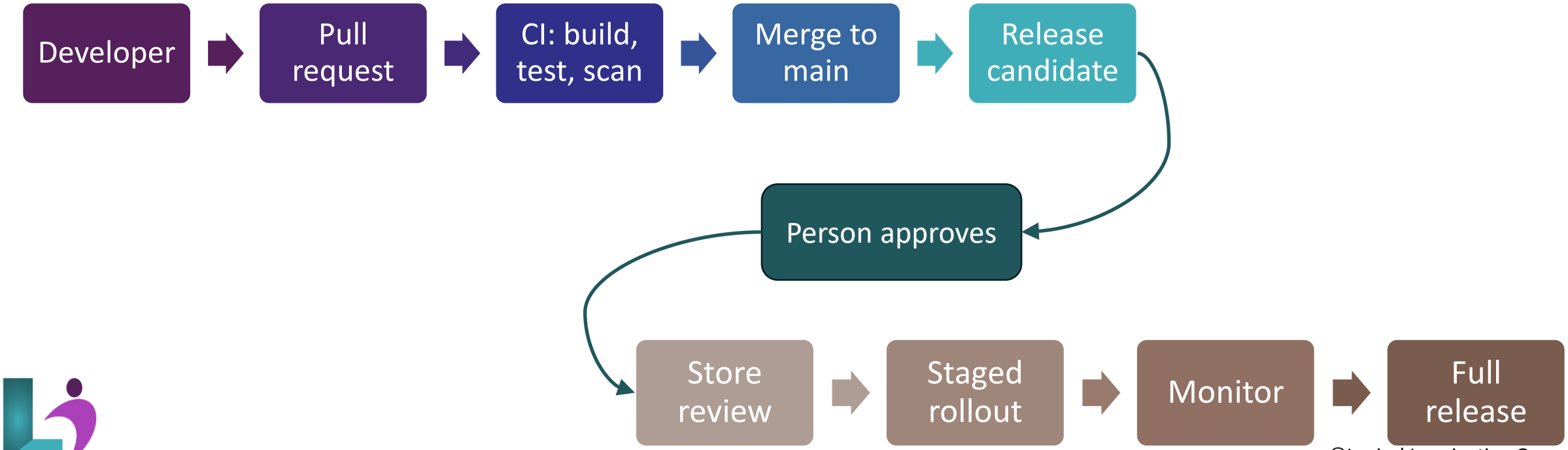
- Explain continuous integration, continuous delivery, and continuous deployment and why mobile differs from web
- Describe the stages of a mobile pipeline and what each one protects
- Model a pipeline as a graph of jobs with dependencies, parallel levels, and path filters
- Read a pipeline's critical path and use it to decide where to speed it up



©Logical Imagination Group LLC

What CI and CD Mean, and Why Mobile Is Different

- Continuous integration: merge small changes often and verify each one automatically
- Continuous delivery: every change that passes is always ready to release; a person decides when
- Continuous deployment: every change that passes is released automatically
- Mobile adds store review, signing, device fragmentation, and users who delay updates



©Logical Imagination Group LLC

One Change, Two Timelines: Web and Mobile

- Web: merge at 10:00, canary by 10:20, all users by 11:00; a bad change is rolled back in minutes
- Mobile: merge at 10:00, release candidate by 10:40, then store review that can take hours to days
- After approval comes a staged rollout over days, and users update when they choose
- A defect that ships can stay on devices for weeks, so prevention beats repair
- Mobile teams compensate with feature flags, server-side kill switches, staged rollouts, and a rehearsed hotfix path



©Logical Imagination Group LLC

The Stages of a Mobile Pipeline

- Validate: lint, static analysis, dependency and secret scans
- Build: compile for each platform and variant; iOS builds need macOS
- Test: unit tests, integration tests, UI tests on emulators or devices
- Package and publish: sign, store the artifact, distribute to testers or the store

Code Bite

```
# Generic shape, not product specific:
validate: [lint, static-analysis,
                  secret-scan]
build:    [android-build, ios-build]
# ios-build needs a macOS runner
test:     [unit, integration, ui-tests]
package:  [sign, publish-artifact]
```



©Logical Imagination Group LLC

Fail Fast: Order Stages by Cost and Failure Rate

- Run cheap, likely-to-fail checks first and expensive ones last
- The same checks in a different order give a very different wait
- A lint error is found at minute 1 instead of minute 37
- Independent checks can also run in parallel

Code Bite

```
# Order A: expensive first
build        10 min
ui-tests     25 min
unit          1 min
lint          1 min   <- fails
# developer learns at minute 37


# Order B: cheapest first
lint          1 min   <- fails
# developer learns at minute 1
# nothing else ran, nothing wasted
```



©Logical Imagination Group LLC

Pipelines as Code: Graphs, Parallelism, and Path Filters

- Define the pipeline in a file that lives with the code and is reviewed like code
- Jobs form a directed graph: a job runs after the jobs it needs
- Independent jobs run in parallel; the critical path sets the minimum time
- Path filters skip jobs a change cannot affect, which matters in monorepos

Code Bite

```
const p = planPipeline([
  {name: "lint", needs: [] }, # 60,000ms
  {name: "unit", needs: [] }, # 180,000ms
  {name: "build", needs: ["lint", "unit"] },
      # 300,000ms
  {name: "deploy", needs: ["build"]}
      # 120,000ms
]);
# levels: [[lint, unit], [build], [deploy]];
# criticalPathMs 600000; totalMs 660000
```



©Logical Imagination Group LLC

Reading a Pipeline File (GitHub Actions Style)

- Triggers say when it runs; path filters limit it to relevant changes
- Each job names the runner it needs; iOS jobs need macOS
- needs creates the edges of the graph
- Syntax differs by product; the ideas do not

Code Bite

```
on:
  pull_request:
    paths: ["android/**", "shared/**"]
jobs:
  lint:
    runs-on: ubuntu-latest
  build:
    needs: [lint]
    runs-on: ubuntu-latest
    steps:
      - run: ./gradlew assembleDebug
# GitHub Actions style; others differ
```



©Logical Imagination Group LLC

Critical Path: A Worked Example

- Earliest finish = a job's duration plus the latest finish among the jobs it needs
- The largest finish time is the critical path
- Slack is how long a job can slip without delaying the run
- Speed up ios by 4 minutes and the run gets faster; speed up android and nothing changes

Code Bite

```
job          min  needs
lint           1  -
unit           3  -
android        8  lint
ios           12  lint
ui-android    15  android
ui-ios        18  ios
sign           2  ui-*, unit
# critical path: ???
```



©Logical Imagination Group LLC

Lesson 1

Skill Check

1. What is the difference between continuous delivery and continuous deployment?

A) None
B) Delivery keeps every passing change releasable and a person decides; deployment releases it automatically
C) Deployment is slower
D) Delivery skips tests

2. Why do iOS builds affect pipeline design?

A) Swift is slow
B) Apple's toolchain needs macOS runners, which are scarcer and costlier
C) They need no tests
D) They cannot be cached

3. What does the critical path of a pipeline tell you?

A) The most important job
B) The shortest possible wall-clock time given unlimited parallelism
C) The slowest developer
D) The number of tests

4. Why order checks from cheapest to most expensive?

A) It looks tidy
B) Most failures are reported quickly before costly stages run
C) It saves disk space
D) Tools require it



©Logical Imagination Group LLC

Lesson 2: Build Automation: Reproducible Builds and Caching

- Explain what makes a build reproducible and why it matters for audit and debugging
- Describe how Gradle and Xcode builds are driven from the command line in a pipeline
- Design a dependency cache with a content-based key and restore keys
- Implement a cache key function and a restore picker



©Logical Imagination Group LLC

Reproducible, Scripted Builds

- The pipeline builds with the same command a developer can run locally
- Pin the toolchain: JDK, Gradle wrapper, Xcode version, and dependency versions
- Commit lock files; avoid floating versions such as latest
- Record the commit, tool versions, and build number with every artifact

Code Bite

```
# Same commands locally and in CI
# (verify flags in current documentation)
./gradlew assembleRelease
# uses the pinned Gradle wrapper
xcodebuild -scheme PNCMobile -destination
   'generic/platform=iOS' archive


Build record stored with the artifact:
  commit 9f2c1ab, build 1087, JDK 17.0.x,
Gradle 8.x, Xcode 16.x
```



©Logical Imagination Group LLC

Why Builds Drift

- Floating versions such as latest or 1.+ pull in tomorrow's code
- Unpinned tools: a runner image update can silently change the JDK or Xcode version
- Build-time clocks, absolute paths, and random IDs end up inside the binary
- Network fetches during a build can return different content each time
- Aim for functional reproducibility first; bit-for-bit identical output is a harder, optional goal

Pro Tips

1. Rebuild a release from six months ago once a quarter, on a clean runner. Every surprise is a gap you found before an auditor or an incident did.



©Logical Imagination Group LLC

Driving Android and iOS Builds from the Command Line

- Android: the Gradle wrapper runs tasks that name what to produce
- iOS: xcodebuild builds, tests, and archives a scheme for a destination
- Publish test results and logs as pipeline artifacts
- Flags and task names can change between releases, so check current documentation

Code Bite

```
# Android
./gradlew test
./gradlew assembleRelease
./gradlew bundleRelease   # .aab

# iOS
xcodebuild test -scheme PNCMobile \
  -destination "$SIM_DEST"
xcodebuild archive -scheme PNCMobile \
  -archivePath build/PNCMobile.xcarchive
# SIM_DEST: set once per runner
```



©Logical Imagination Group LLC

Caching Dependencies and Build Outputs

- Downloading dependencies and recompiling unchanged code dominate many mobile builds
- Cache the dependency directories and, where supported, the build cache
- A cache key must change exactly when the inputs that matter change
- Restore keys let a near match seed the cache when the exact key is missing

Code Bite

```
export function cacheKey(i: KeyInput): string {
  const parts =
      Object.keys(i.files).sort().map((p) => p +
      ":" + sha(i.files[p]));
  // path and content, in a fixed order
  return [i.prefix, i.os, i.toolchain,
      sha(parts.join("\n")).slice(0,
      16)].join("-");
}
// gradle-linux-jdk17-3fa9c2e41b07d5aa; restore
// keys: gradle-linux-jdk17-, gradle-linux-
```



©Logical Imagination Group LLC

Cache Math and Cache Pitfalls

- A cold build downloads for about 6 minutes; a warm cache restores in about 30 seconds
- At 40 runs a day that is roughly 220 runner-minutes saved
- Pitfalls: stale entries, oversized caches, and sharing across unlike OS or toolchain
- A cache is an input you did not review, so treat it as untrusted

Code Bite

```
cold download    6.0 min
cache restore    0.5 min
saved per run    5.5 min
x 40 runs a day  = 220 min
# about 3.7 runner-hours daily


# Cache hygiene
key on content, OS, toolchain
cap the size, expire old entries
never cache secrets or signing files
keep PR caches away from release
```



©Logical Imagination Group LLC

Build Variants, Numbers, and Where Builds Run

- Android build types and flavors and iOS schemes and configurations create debug, staging, and release variants
- Build once per commit and reuse the artifact; do not rebuild for each environment
- Build numbers must increase for every upload; derive them from the pipeline, not from a person
- Choose hosted or self-hosted runners with security, cost, and macOS capacity in mind

Code Bite

Variant	Points at	Signed with	Use
debug	dev services	debug key	developers
staging	test services	test signing	QA and UAT
release	production services	release signing	customers

```
# Build release ONCE per commit;
# promote the same artifact to staging,
# then production.
```



©Logical Imagination Group LLC

Hosted or Self-Hosted Runners?

- Hosted: no maintenance and a quick start, but shared infrastructure and less control
- Self-hosted: control over network, tools, and secrets, but you own patching and capacity
- macOS capacity is the usual iOS bottleneck
- Many banks self-host inside controlled networks

Code Bite

Question	Hosted	Self-hosted
Setup effort	low	high
Patching	vendor	you
Network control	limited	full
Data residency	vendor	yours
macOS capacity	queue	buy Macs
Cost shape	per min	fixed + ops

```
# Prefer fresh (ephemeral) runners:
# one job, then wipe the machine
```



©Logical Imagination Group LLC

Lesson 2

Skill Check

1. What makes a build reproducible?

A) Running it twice
B) Pinned toolchain and locked dependencies, so the same source gives the same result anywhere
C) A fast machine
D) A large cache

2. What should a cache key be based on?

A) The date
B) A hash of the inputs that determine the cached content, plus OS and toolchain
C) The developer's name
D) The branch name only

3. What is a restore key for?

A) Encrypting the cache
B) Seeding a build from a near-match cache when the exact key is missing
C) Deleting old caches
D) Signing the build

4. Why derive build numbers in the pipeline?

A) People forget
B) Stores require every upload to have a higher number, and automation avoids mistakes
C) It is faster to type
D) Numbers must be random

©Logical Imagination Group LLC

Lesson 3: Automated Testing in the Pipeline and Quality Gates

- Describe the test pyramid and where each kind of mobile test runs in a pipeline
- Explain flaky tests, why they erode trust, and how quarantine keeps them visible
- Shard a test suite to cut wall-clock time
- Implement sharding, flaky detection, and a quality gate
- Time: 90 minutes | Lecture, live demo, self-guided exercise, knowledge check



©Logical Imagination Group LLC

The Test Pyramid in a Pipeline

- Many fast unit tests, fewer integration tests, few end-to-end UI tests
- Unit tests run on every commit; UI tests often run on merge or nightly
- Emulators, simulators, and device farms trade cost against realism
- Fast tests belong early so feedback is quick

Code Bite

```
      /\        UI, end-to-end
     /  \       few, slow, nightly
    /----\      Integration
   /      \     some, minutes, fakes
  /--------\    Unit
                many, seconds, every commit
```



©Logical Imagination Group LLC

Which Test Catches Which Defect?

- Pick the cheapest test that can catch the defect
- Pure logic belongs in unit tests
- Boundaries between components belong in integration or contract tests
- Only a real screen, OS, and device can answer some questions

Code Bite

```
Rounding bug in interest calc
  -> unit test (milliseconds)
API renames a response field
  -> integration / contract test
Login crashes on a small screen
  -> UI test, device matrix
New OS changes a permission prompt
  -> UI test, real device
Slow cold start
  -> performance test, real device
```



©Logical Imagination Group LLC

Flaky Tests and Quarantine

- A flaky test passes and fails on the same code
- Flakiness teaches developers to ignore red builds, which is the real damage
- Detect: the same test both failed and passed on the same commit
- Quarantine: stop it blocking merges, keep running and reporting it, assign an owner and a deadline

Code Bite

```
// flaky = failed and passed on the SAME
// commit
export function detectFlaky(history:
    RunRecord[]): string[] {
  // group by commit and test; a group that
  // contains both a pass and a fail is
  // flaky
}
// transfer failed on c2 and passed on c3:
// a different commit, so a real fix, not
// flakiness
```



©Logical Imagination Group LLC

Fixing a Flaky Test

- Common causes: fixed sleeps, shared state, real network calls, test order, clock and timezone
- Wait for a condition, not for a duration
- Reset state in setup and replace network calls with fakes
- Inject the clock so time is under the test's control

Code Bite

```
// Flaky: hopes 500ms is enough
await sleep(500);
expect(screen.hasBalance()).toBe(true);

// Stable: waits for the condition
await waitFor(
  () => screen.hasBalance(),
  { timeoutMs: 5000 });
expect(screen.balance())
  .toBe("$120.00");
```



©Logical Imagination Group LLC

Sharding and Quality Gates

- Sharding splits the suite across parallel runners so wall-clock time falls
- Balance by historical duration, longest first, not by test count
- A quality gate is an automatic decision: coverage, failures, static analysis, and security results
- Gates block on new problems and are tuned so people trust them

Code Bite

```
shardTests([a:90, b:80, c:70, d:60, e:50,
    f:40], 3)
// shard 1: a, f = 130 ms    shard 2: b, e
// = 130 ms    shard 3: c, d = 130 ms
// one runner would need 390 ms; three
// balanced shards need 130 ms
```



©Logical Imagination Group LLC

Greedy Sharding, Step by Step

- Sort tests longest first
- Give each test to the shard with the least work so far
- The slowest shard sets the wall-clock time
- No sharding can beat the longest single test, so split a huge test

Code Bite

```
tests: a90 b80 c70 d60 e50 f40
shards: 3

a90 -> S1   (90)
b80 -> S2   (80)
c70 -> S3   (70)
d60 -> S3   (70 + 60 = 130)
e50 -> S2   (80 + 50 = 130)
f40 -> S1   (90 + 40 = 130)

# slowest shard 130 vs 390 serial
```



©Logical Imagination Group LLC

A Quality Gate in Practice

- A gate lists its rules and reports every failure, not just the first
- Judge new and changed code so old debt does not block everyone
- Ratchet the baseline so coverage never quietly falls
- Allow a logged, time-limited override with a named owner

Code Bite

```
gate: pull-request
  failing tests       == 0
  new-code coverage   >= 80%
  total coverage      >= baseline
  new static issues   == 0
  critical vulns      == 0

result: BLOCKED
  - new-code coverage 62% < 80%
  - 2 new static issues
```



©Logical Imagination Group LLC

Lesson 3

Skill Check ✓

1. Why should most tests be unit tests?

A) They are more realistic
B) They are fast, cheap, and stable
C) They need devices
D) They are required by stores

2. How can you detect a flaky test from history?

A) It is slow
B) It both failed and passed on the same commit
C) It fails once
D) It uses the network

3. How should tests be balanced across shards?

A) By count
B) By historical duration, longest first
C) Alphabetically
D) Randomly

4. What should a quarantined test do?

A) Be deleted
B) Keep running and reporting without blocking merges, with an owner and deadline
C) Be skipped silently
D) Block everything



©Logical Imagination Group LLC